

NYCU Introduction to Machine Learning, Homework 2

[112550069], [謝侑哲]

Part. 1, Coding (60%):

(25%) Logistic Regression w/ Gradient Descent Method

1. (5%) Show the hyperparameters (learning rate and iteration, etc) that you used and the weights and intercept of your model.

```
LR = LogisticRegression(  
    learning_rate=10, # You  
    num_iterations=100000,  
)
```

```
2024-10-22 14:24:48.881 | INFO | __main__:main:209 - LR: Weights: [ -7.68298511 -4.03227128 37.98947173 -29.19871582 -11.43556327],  
Intercept: -66.17492818543947
```

2. (5%) Show the AUC of the classification results on the testing set.

```
2024-10-22 14:24:48.882 | INFO | __main__:main:210 - LR: Accuracy=0.8095, AUC=0.8216
```

3. (15%) Show the accuracy score of your model on the testing set

```
2024-10-22 14:24:48.882 | INFO | __main__:main:210 - LR: Accuracy=0.8095, AUC=0.8216
```

(25%) Fisher Linear Discriminant, FLD

4. (5%) Show the mean vectors m_i ($i=0, 1$) of each class, the within-class scatter matrix S_w , and the between-class scatter matrix S_b of the training set.

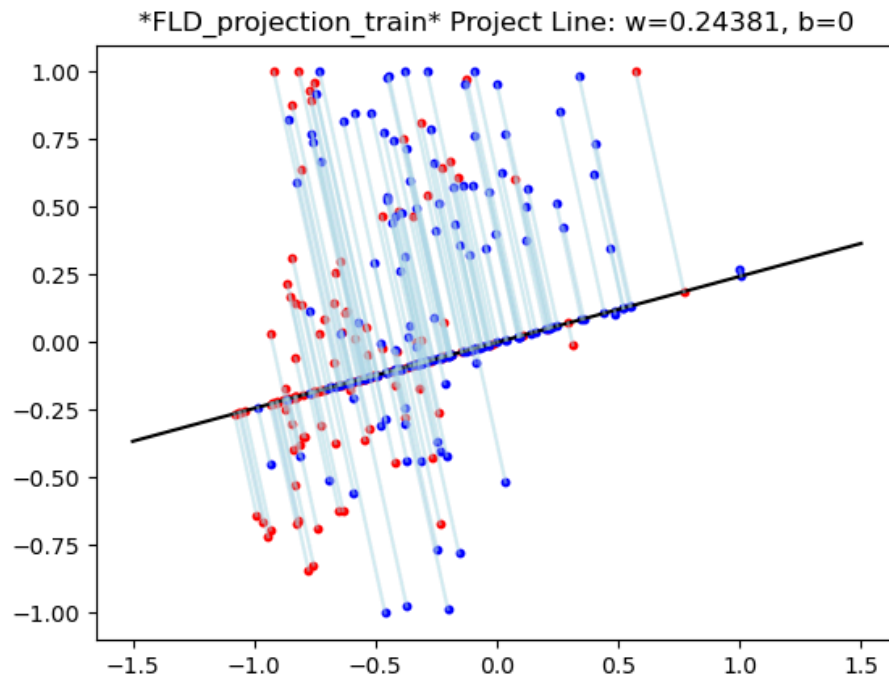
```
2024-10-29 20:52:18.291 | INFO | __main__:main:233 - FLD: m0=[-0.27747695 0.29565197], m1=[-0.58535466 0.02331584] of cols=['10', '20'  
]  
2024-10-29 20:52:18.292 | INFO | __main__:main:234 - FLD:  
Sw=  
[[17.17974856 5.44299487]  
 [ 5.44299487 44.81848741]]  
2024-10-29 20:52:18.294 | INFO | __main__:main:235 - FLD:  
Sb=  
[[0.09478869 0.08384622]  
 [0.08384622 0.07416696]]
```

5. (5%) Show the Fisher's linear discriminant w of the training set.

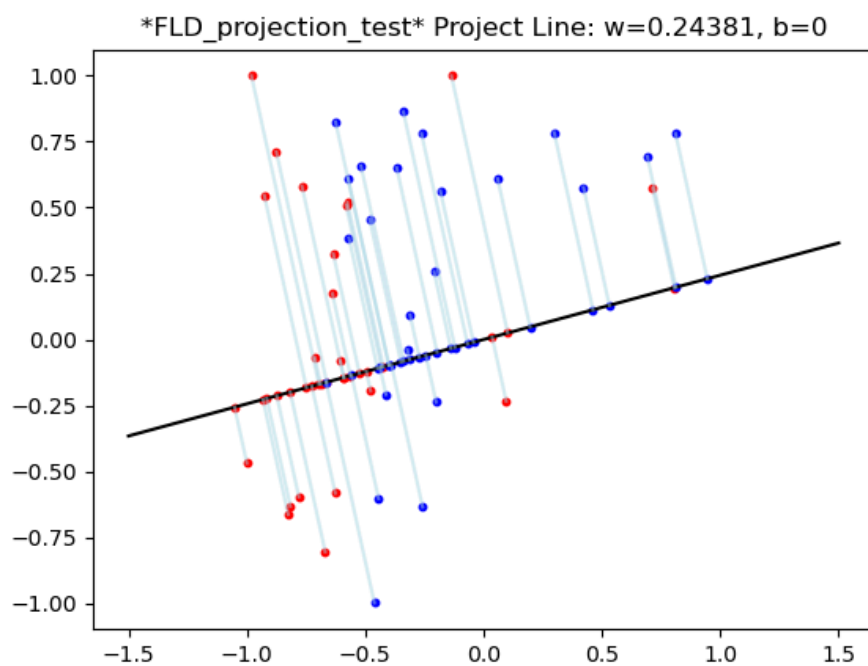
```
2024-10-22 14:24:48.965 | INFO | __main__:main:234 - FLD:  
w=  
[-0.0166359 -0.00405607]
```

6. (15%) Obtain predictions for the testing set by measuring the distance between the projected value of the testing data and the projected means of the training data for the two classes. (Also, plot for training data). Show the accuracy score on the testing set.

Training Dataset:



Testing Dataset:



```
2024-10-22 14:24:48.966 | INFO | __main__:main:235 - FLD: Accuracy=0.7619
```

(10%) Code Check and Verification

7. (10%) Lint the code and show the PyTest results.

```
(AIHomework) C:\Users\money\桌面\學校課業資料夾\大二上\ML\NYCU-Intro-ML\hw2>flake8 main.py
```

```
(AIHomework) C:\Users\money\桌面\學校課業資料夾\大二上\ML\NYCU-Intro-ML\hw2>
```

```
(AIHomework) C:\Users\money\桌面\學校課業資料夾\大二上\ML\NYCU-Intro-ML\hw2>pytest ./test_main.py -s
===== test session starts =====
platform win32 -- Python 3.8.18, pytest-7.4.4, pluggy-1.0.0
rootdir: C:\Users\money\桌面\學校課業資料夾\大二上\ML\NYCU-Intro-ML\hw2
plugins: anyio-4.4.0
collected 2 items

test_main.py (395, 2) (395,)
2024-10-29 21:08:40.270 | INFO | test_main:test_logistic_regression:35 - accuracy=0.9379
.(395, 2) (395,)
2024-10-29 21:08:40.296 | INFO | test_main:test_fld:45 - accuracy=0.8759
+
===== 2 passed in 4.11s =====
```

Part. 2, Questions (40%):

1. (10%)

- Is logistic 'regression' used for regression problems?
- If not, what task is it primarily used? (without any additional techniques and modification); If yes, how can it be implemented?
- Why are we using the logistic function in such a task? (list two reasons)
- If there are multi-class, what should we use to substitute it?

1. Logistic regression is not used for regression problems.

2. It predict the probability of classes, so it is used for classification problem

3. 1) Logistic regression is more explainable because it outputs the probability of classes rather than directly classifying with output > 0 . 2) Loss function of logistic regression (cross-entropy) is better than MSE (general linear model use) because it will not be influenced by outliers that easily. 3) sigmoid can bring more non-linear properties than a general linear model.

4. To do multi-class classification tasks, we should substitute the sigmoid with softmax function which will be multinomial logistic regression.

2. (15%) When a trained classification model shows exceptionally high precision but unusually low recall and F1-score, what potential issues might arise? How can these issues be resolved? List at least three solutions.

High precision means when the model predicts one data is positive, it is much likely positive. While low recall means when the model predicts one data which is truly positive, it is very probable that the model gives a false prediction. Low F1-score means the model's precision and recall are not balanced.

The potential issue is that the model is unable to correctly recognize positive cases. It may be a big problem in security or other tasks. For example, a model will recognize malware as positive. With low recall and F1-score, it will miss a lot of malware, one of each may destroy the whole system.

Solution:

1. One reason for this situation is that the positive and negative rate of data is skewed, and negative data is too much. The problem will cause the model to guess negatively. To solve it, we can try data augmentation or other techniques to make the data's labels more balanced.

2. We can adjust the loss function, like enhancing the weight of positive data terms. This way, the model will pay more attention to make the correct classification of one's ground truth positive. For example: $L(x) = w * p(x)\log(q(x))$ when $p(x) = 1$

3. We can also directly adjust the classification threshold without retrain it. For example, we can adjust the threshold from 0.5 to 0.4, which model will be more probable to classify data to positive increasing recall.

3. (15%) In this homework, we use Cross-Entropy as the loss function for Logistic Regression. Can we use Mean Square Error (MSE) instead? Why or why not? Please explain in detail.

Cross-Entropy means that the model has how much uncertainty to the truth probability. While we use MSE to replace Cross-Entropy, the loss function will be like $L(x) = (p(x)-q(x))^2$, which is the square of difference between probability of ground truth and prediction.

Although this kind of loss function is still able to model the error distance from prediction to ground truth. However, its performance is not as good as Cross-Entropy, so MSE cannot be used.

There are reasons that the MSE is not better than Cross-Entropy:

1. MSE's penalty to the model is not sufficient. For example, a class 1 data $p(x) = 1$, model predict $q(x) = 0.0001$, the Cross-Entropy ≈ 13 , MSE ≈ 1 . After some optimization, the model predicts $q(x) = 0.01$, Cross-Entropy ≈ 6 , MSE ≈ 1 . The probability significantly increases to 100 times, and the Cross-Entropy has a large change. However, the MSE is almost the same even with a significant optimization. We can be aware that MSE is not as good as Cross-Entropy when it gives a sufficient penalty to the model.

2. Cross-Entropy of logistic regression is convex because $(d^2 L / d w^2) > 0$, while MSE's 2nd derivative is not always greater or equal than 0 and it is not convex. If we want to find the best solution, the loss function should be convex. Otherwise, if it is not convex, it may get local minimum loss rather than global minimum. Thus, MSE will be more difficult to find the best solution.